OOP_26 Metal Slug — Final UML

«struct»

coordSyst

+x : float
+y : float

«struct»
Box

+x : float
+y : float
+w : float
+h : float

«struct»
Crater

+x : int
+y : int
+width : int
+depth : int
+exposureTime : float

«struct»

FingerprintParam

+id : char[16]
+category : char[32]
+name : char[64]
+value : float
+weight : float
+minVal : float
+maxVal : float
+unit : char[16]
+dataType : char[16]
+description : char[128]

«struct»
RunRecord

+midpointScore : int
+finalScore : int
+timestamp : int

«struct»

Snapshot

+entityStates : char*
+dataSize : int
+timestamp : float
+score : int

«struct»
TimelineNode

+snapshot : Snapshot*
+parent : TimelineNode*
+children : TimelineNode**
+childCount : int
+nodeId : int

«struct»

DeathFingerprint

+playerHealth : int
+playerPosition : coordSyst
+activeWeaponId : int
+enemyConfig : int*
+enemyCount : int
+timeSinceCheckpoint : float

«struct»
ConnectionGene

+inNode : int
+outNode : int
+weight : float
+enabled : bool
+innovation : int

Direction

+virtual ~Direction() = default
+getXSign() : int
+flip() : Direction*

LeftDirection

+getXSign() : int
+flip() : Direction*

RightDirection

+getXSign() : int
+flip() : Direction*

Tile

#gridX : int
#gridY : int

+virtual ~Tile() = default
+isSolid() : bool
+isDestructible() : bool
+isWater() : bool
+virtual onEntityEnter(e:Entity*) : void
+virtual onExplosion(blastRadius:float) : void

AirTile

+isSolid() : bool
+isDestructible() : bool
+isWater() : bool

SolidTile

+isSolid() : bool
+isDestructible() : bool
+isWater() : bool
+onExplosion(blastRadius:float) : void

WaterTile

+isSolid() : bool
+isDestructible() : bool
+isWater() : bool
+onEntityEnter(e:Entity*) : void

IndestructibleTile

+isSolid() : bool
+isDestructible() : bool
+isWater() : bool

Biome

#minAltitude : int
#maxAltitude : int

+virtual ~Biome() = default
+biomeName() : char*
+canHostEnemy(e:Enemy*) : bool
+canHostVehicle(v:Vehicle*) : bool
+ambientSoundId() : int

AerialBiome

+biomeName() : char*
+canHostEnemy(e:Enemy*) : bool
+canHostVehicle(v:Vehicle*) : bool
+ambientSoundId() : int

PlainsBiome

+biomeName() : char*
+canHostEnemy(e:Enemy*) : bool
+canHostVehicle(v:Vehicle*) : bool
+ambientSoundId() : int

AquaticBiome

-seaLevel : int

+biomeName() : char*
+canHostEnemy(e:Enemy*) : bool
+canHostVehicle(v:Vehicle*) : bool
+ambientSoundId() : int

DamageState

#owner : DamagableEntity*
#remainingTime : float

+virtual ~DamageState() = default
+enter(d:DamagableEntity*) : void
+exit(d:DamagableEntity*) : void
+update(d:DamagableEntity*, dt:float) : void
+hueAlpha() : float
+next() : DamageState*

NormalDamageState

+enter(d:DamagableEntity*) : void
+exit(d:DamagableEntity*) : void
+update(d:DamagableEntity*, dt:float) : void
+hueAlpha() : float
+next() : DamageState*

InjuredDamageState

+enter(d:DamagableEntity*) : void
+exit(d:DamagableEntity*) : void
+update(d:DamagableEntity*, dt:float) : void
+hueAlpha() : float
+next() : DamageState*

CriticalDamageState

+enter(d:DamagableEntity*) : void
+exit(d:DamagableEntity*) : void
+update(d:DamagableEntity*, dt:float) : void
+hueAlpha() : float
+next() : DamageState*

DeadDamageState

+enter(d:DamagableEntity*) : void
+exit(d:DamagableEntity*) : void
+update(d:DamagableEntity*, dt:float) : void
+hueAlpha(): float
+next() : DamageState*

DamageSource

#cause : Entity*

+virtual ~DamageSource() = default
+amount() : int
+virtual canKillMummy() : bool
+virtual isExplosive() : bool

BulletDamage

+amount() : int
+canKillMummy() : bool

ExplosionDamage

-radius : float

+amount() : int
+canKillMummy() : bool
+isExplosive() : bool

FireDamage

+amount() : int
+canKillMummy() : bool

MeleeDamage

+amount() : int

ContactDamage

+amount() : int

GameMode

+virtual ~GameMode() = default
+setupLevels(lm:LevelManager&) : void
+isComplete(ps:PlayState&) : bool
+onLevelCleared(ps:PlayState&) : void
+modeName() : char*

SurvivalMode

+setupLevels(lm:LevelManager&) : void
+isComplete(ps:PlayState&) : bool
+onLevelCleared(ps:PlayState&) : void
+modeName() : char*

CampaignMode

+setupLevels(lm:LevelManager&) : void
+isComplete(ps:PlayState&) : bool
+onLevelCleared(ps:PlayState&) : void
+modeName() : char*

SelfPlayingMode

-neat : NEATManager*

+setupLevels(lm:LevelManager&) : void
+isComplete(ps:PlayState&) : bool
+onLevelCleared(ps:PlayState&) : void
+modeName() : char*

VehicleModeStrategy

#owner : AmphibiousSlug*

+virtual ~VehicleModeStrategy() = default
+fire(aim:coordSyst) : void
+update(tm:TileMap&, dt:float) : void
+canSwitchTo(tm:TileMap&) : VehicleModeStrategy*

NormalSlugMode

+fire(aim:coordSyst) : void
+update(tm:TileMap&, dt:float) : void
+canSwitchTo(tm:TileMap&) : VehicleModeStrategy*

FlyerMode

-missilesRemaining : int

+fire(aim:coordSyst) : void
+update(tm:TileMap&, dt:float) : void
+canSwitchTo(tm:TileMap&) : VehicleModeStrategy*

MarinerMode

-horizontalMissiles : int
-verticalMissiles : int
-reverseArcMissiles : int

+fire(aim:coordSyst) : void
+update(tm:TileMap&, dt:float) : void
+canSwitchTo(tm:TileMap&) : VehicleModeStrategy*

BehaviourLabel

+virtual ~BehaviourLabel() = default
+labelName() : char*
+matches(window:PlayerInputRecord*, n:int) : float

PressingLabel

+labelName() : char*
+matches(window:PlayerInputRecord*, n:int) : float

ConsolidatingLabel

+labelName() : char*
+matches(window:PlayerInputRecord*, n:int) : float

RetreatingLabel

+labelName() : char*
+matches(window:PlayerInputRecord*, n:int) : float

BurningResourcesLabel

+labelName() : char*
+matches(window:PlayerInputRecord*, n:int) : float

StallingLabel

+labelName() : char*
+matches(window:PlayerInputRecord*, n:int) : float

PlayerAction

#timestamp : float
#aimAngle : float
#positionAtAction : coordSyst

+virtual ~PlayerAction() = default
+apply(p:PlayerSoldier*) : void
+actionId() : int
+clone() : PlayerAction*

IdleAction MoveLeftAction MoveRightAction JumpAction ShootAction GrenadeAction MeleeAction SwitchWeaponAction SwitchCharacterAction

«struct»

PlayerInputRecord

+action : PlayerAction*
+timestamp : float
+aimAngle : float
+positionAtAction : coordSyst

Instruction

#targetVarId : int
#arg : int

+virtual ~Instruction() = default
+execute(ps:PlayState&) : void
+mnemonic() : char*

ManfiInstruction BarabarInstruction ChhapoInstruction JamaInstruction

Node

#id : int
#activationValue : float

+virtual ~Node() = default
+activate(weightedSum:float) : float
+nodeKind() : char*
+clone() : Node*

InputNode HiddenNode OutputNode BiasNode

«struct»

EnemySpawn

+masterCopy : Enemy*
+spawnPosition : coordSyst
+batchSize : int
+biome: Biome*

Grenade

#blastRadius : float
#damage : int

+virtual ~Grenade() = default
+clone() : Grenade*
+spawnProjectile(owner:Entity*, aim:coordSyst) : Projectile*

HandGrenadePrototype

+clone() : Grenade*
+spawnProjectile(owner:Entity*, aim:coordSyst) : Projectile*

FireBombPrototype

+clone() : Grenade*
+spawnProjectile(owner:Entity*, aim:coordSyst) : Projectile*

Game

-window : sf::RenderWindow*
-clock : sf::Clock
-stateMgr : GameStateManager*
-isRunning : bool

+run() : void
+processEvents() : void
+update(dt:float) : void
+render() : void
+shutdown() : void

GameStateManager

-states : GameState**
-stateCount : int
-capacity : int

+pushState(s:GameState*) : void
+popState() : void
+replaceTop(s:GameState*) : void
+currentState() : GameState*
+update(dt:float) : void
+render(rw:sf::RenderWindow&) : void

GameState

#owner : GameStateManager*

+virtual ~GameState() = default
+enter() : void
+exit() : void
+update(dt:float) : void
+render(rw:sf::RenderWindow&) : void
+handleInput(in:InputHandler&) : void

MenuState

-menu : MainMenu*

+enter() : void
+exit() : void
+update(dt:float) : void
+render(rw:sf::RenderWindow&) : void
+handleInput(in:InputHandler&) : void

PlayState

-entityMgr : EntityManager*
-levelMgr : LevelManager*
-players : PlayerSoldier*[4]
-numPlayers : int
-activePlayerIdx: int
-mode : GameMode*
-developerModeActive : bool
-collisionMgr : CollisionManager
-scoreMgr : ScoreManager
-hud : HUD*
-reservePool : EnemyReservePool
-camera : Camera
-input : InputHandler
-rainSystem : RainSystem
-timelineMgr : TimelineManager
-fingerprint : FingerprintRecorder
-autopilot : FingerprintAutopilot*
-deathComp : DeathCompositor
-rageBait : RagebaitEngine
-chainOfDeath : ChainOfDeath
-spawner : EnemySpawner
-sound : SoundManager
-voice : VoiceActivationEngine
-terminal : UrduAssemblyTerminal

+enter() : void
+exit() : void
+update(dt:float) : void
+render(rw:sf::RenderWindow&) : void
+handleInput(in:InputHandler&) : void
+switchCharacter() : void
+getActivePlayer() : PlayerSoldier*
+getScoreManager() : ScoreManager&
+advanceLevel() : void
+toggleDeveloperMode() : void
+spawnProjectile(p:Projectile*) : void
+spawnExplosion(ex:Explosion*) : void
+setMode(m:GameMode*) : void

EntityManager

-entities : Entity**
-entityCount : int
-capacity : int

+addEntity(e:Entity*) : void
+removeEntity(e:Entity*) : void
+growRoster() : void
+updateAll(tm:TileMap&, dt:float) : void
+renderAll(rw:sf::RenderWindow&, cam:Camera&) : void
+purgeDead() : void
+getEntities() : Entity**
+getCount() : int

LevelManager

-currentLevel : Level*
-currentLevelIdx : int

+loadLevel(idx:int, mode:GameMode*) : void
+unloadCurrent() : void
+advance() : void
+loadBossLevel() : void
+getCurrent() : Level*

Entity

#position : coordSyst
#velocity : coordSyst
#width : float
#height : float
#isAlive : bool
#isOnGround : bool
#touchesWall : bool

+virtual ~Entity() = default
+update(tm:TileMap&, dt:float) : void
+render(rw:sf::RenderWindow&) : void
+checkCollision(other:Entity*) : bool
+getHitbox() : Box

DamagableEntity

#healthPoints : int
#maxHealthPoints : int
#isInvulnerable : bool
#currentDamageState : DamageState*

+virtual takeDamage(amt:int, src:DamageSource*) : void
+virtual die() : void
+advanceDamageState() : void
+getDamageState() : DamageState*

Damage staging is owned here as a swappable
DamageState* (NormalDamageState -> InjuredDamageState
-> CriticalDamageState -> DeadDamageState).
Spec: "Same damage taking applies to player vehicles."

Soldier

#currentTransformState : TransformationState*
#facing : Direction*
#baseSpeed : float
#baseFireRate : float
#attackDamage : int
#aimAngle : float

+changeState(s:TransformationState*) : void
+virtual move(dt:float) : void
+virtual attack() : void
+virtual jump() : void

PlayerSoldier

#lives : int
#saturationStat : int
#canMelee : bool
#fireRateMultiplier : float
#speedMultiplier : float
#meleeDamageMultiplier : float
#ammoPickupMultiplier : float
#blastRadiusMultiplier : float
#powerUpTimer : float
#powerUpActive : bool
#fusionCooldownTimer : float
#fusionUsedInLevel : bool

+specialPowerUp() : void
+respawn() : void
+virtual throwGrenade() : void
+virtual meleeAttack() : void
+pickupWeapon(w:Weapon*) : void
+virtual takeDamage(amt:int, src:DamageSource*) : void
+virtual onVehicleDestroyed() : bool
+spawnFusionCompanion(members:PlayerSoldier**, count:int) : FusionCompanion*
+checkSaturationTransition() : void
+getGrenadePrototype() : Grenade*

Marco

+specialPowerUp() : void
+attack() : void
+meleeAttack() : void

Tarma

+specialPowerUp() : void
+onVehicleDestroyed() : bool

Eri

+specialPowerUp() : void
+throwGrenade() : void
+meleeAttack() : void

Fio

+specialPowerUp() : void
+pickupWeapon(w:Weapon*) : void

FusionCompanion

-linkedPlayer : PlayerSoldier*
-fusionMembers : PlayerSoldier**
-memberCount : int
-remainingDuration : float
-averagedFireRate : float
-averagedSpeed : float
-averagedDamage : int

+follow() : void
+update(tm:TileMap&, dt:float) : void
+attack() : void
+takeDamage(amt:int, src:DamageSource*) : void

GhostPlayer

-damageMultiplier : float
-healthMultiplier : float
-styleProfile : FingerprintRecorder*
-autopilot : FingerprintAutopilot*

+takeDamage(amt:int, src:DamageSource*) : void
+autoPlayFromProfile() : void
+attack() : void
+update(tm:TileMap&, dt:float) : void

TransformationState

#owner : Soldier*
#duration : float

+virtual ~TransformationState() = default
+enter(s:Soldier*) : void
+exit(s:Soldier*) : void
+update(s:Soldier*, dt:float) : void

NormalTransformState

+enter(s:Soldier*) : void
+exit(s:Soldier*) : void
+update(s:Soldier*, dt:float) : void

FatTransformState

-fannumRadius : float
-isBulletMagnet : bool

+enter(s:Soldier*) : void
+exit(s:Soldier*) : void
+update(s:Soldier*, dt:float) : void
+getMagnetism(out_radius:float&, out_active:bool&) : void

MummyTransformState

+enter(s:Soldier*) : void
+exit(s:Soldier*) : void
+update(s:Soldier*, dt:float) : void

UndeadTransformState

+enter(s:Soldier*) : void
+exit(s:Soldier*) : void
+update(s:Soldier*, dt:float) : void

Vehicle

#pilot : PlayerSoldier*
#isOccupied : bool
#fireRate : float

+enter(p:PlayerSoldier*) : void
+exit() : PlayerSoldier*
+virtual onDestroy() : void
+fire(aim:coordSyst) : void
+virtual takeDamage(amt:int, src:DamageSource*) : void

GroundVehicle

#treadSpeed : float

+virtual traverseStep(tm:TileMap&) : void

AerialVehicle

#hoverHeight : float

+virtual ascend(dt:float) : void

AquaticVehicle

#depth : float

+virtual dive(dt:float) : void

MetalSlug

+fire(aim:coordSyst) : void
+onDestroy() : void

SlugFlyer

-missilesRemaining : int

+fire(aim:coordSyst) : void
+onDestroy() : void

SlugMariner

-horizontalMissiles : int
-verticalMissiles : int
-reverseArcMissiles : int

+fire(aim:coordSyst) : void
+onDestroy() : void

AmphibiousSlug

-currentMode : VehicleModeStrategy*

+switchMode(tm:TileMap&) : void
+fire(aim:coordSyst) : void
+onDestroy() : void

EnemyVehicle

#movesHorizontally : bool
#scoreValue : int

+virtual onDestroy() : void
+die() : void
+getScoreValue() : int

FlyingTara

+attack() : void
+onDestroy() : void

M15ABradley

-isStationary : bool

+attack() : void
+onDestroy() : void

EnemySub

+attack() : void
+onDestroy() : void

Enemy

#scoreValue : int
#isGrudgeful : bool
#currentAIState : EnemyAIState*
#pendingDrop : Collectible*

+attack() : void
+virtual update(tm:TileMap&, dt:float) : void
+virtual render(rw:sf::RenderWindow&): void
+virtual onDeath() : void
+die() : void
+virtual onPlayerContact(p:PlayerSoldier*) : void
+virtual blocksProjectile(incomingDir:coordSyst, isExplosive:bool) : bool
+virtual isAerial(): bool
+pathToPlayer(tm:TileMap&, target:coordSyst) : void
+changeAIState(s:EnemyAIState*) : void
+clone() : Enemy*
+getScoreValue() : int
+getIsGrudgeful() : bool
+takePendingDrop() : Collectible*

Polymorphic dispatch entry-points lifted to Enemy
(onPlayerContact, blocksProjectile, isAerial) so
CollisionManager / ScoreManager dispatch through
Enemy* without dynamic_cast (P1 invariant).
Default behaviours are inert; subclasses override
only what they customise.

EnemyAIState

#owner : Enemy*

+virtual ~EnemyAIState() = default
+enter(e:Enemy*) : void
+exit(e:Enemy*) : void
+update(e:Enemy*, tm:TileMap&, dt:float) : void

PatrolAIState

+enter(e:Enemy*) : void
+exit(e:Enemy*) : void
+update(e:Enemy*, tm:TileMap&, dt:float) : void

ChaseAIState

+enter(e:Enemy*) : void
+exit(e:Enemy*) : void
+update(e:Enemy*, tm:TileMap&, dt:float) : void

AttackAIState

+enter(e:Enemy*) : void
+exit(e:Enemy*) : void
+update(e:Enemy*, tm:TileMap&, dt:float) : void

GrudgefulAIState

+enter(e:Enemy*) : void
+exit(e:Enemy*) : void
+update(e:Enemy*, tm:TileMap&, dt:float) : void

EnemyReservePool

-reservedEnemies : Enemy**
-reserveCount : int
-capacity : int
-counterattackTimer : float

+addToReserve(e:Enemy*) : void
+triggerCounterattack() : void
+update(dt:float) : void
+clearPool() : void
+purgeDeadReferences() : void

Infantry

#batchId : int

+getBatchId() : int

RebelSoldier

+update(tm:TileMap&, dt:float) : void
+attack() : void
+clone() : Enemy*

ShieldedSoldier

+update(tm:TileMap&, dt:float) : void
+blockAttack(bulletDirection:coordSyst) : bool
+blocksProjectile(incomingDir:coordSyst, isExplosive:bool) : bool
+attack() : void
+takeDamage(amt:int, src:DamageSource*) : void
+clone() : Enemy*

BazookaSoldier

+update(tm:TileMap&, dt:float) : void
+attack() : void
+clone() : Enemy*

Grenadier

+update(tm:TileMap&, dt:float) : void
+attack() : void
+clone() : Enemy*

Paratrooper

+enum InfantryKind {KIND_REBEL, KIND_SHIELDED, KIND_BAZOOKA, KIND_GRENADIER}
-hasLanded : bool
-isParachuting : bool
-kind : InfantryKind
-landedSpawn : Infantry*

+update(tm:TileMap&, dt:float) : void
+attack() : void
+onLand() : void
+onDeath() : void
+clone() : Enemy*
+takeLandedSpawn() : Infantry*
+hasParachuted() : bool
+isAerial() : bool

Undead

+onPlayerContact(p:PlayerSoldier*) : void

Re-abstracts Enemy::onPlayerContact (which has
a virtual default no-op). Forces every Undead
child to define a transformation behaviour:
MummyWarrior -> Mummy state, Zombie -> Undead state.

MummyWarrior

-crumbleTimer : float
-isCrumbled : bool
-resurrectionDelay : float

+update(tm:TileMap&, dt:float) : void
+attack() : void
+onPlayerContact(p:PlayerSoldier*) : void
+takeDamage(amt:int, src:DamageSource*) : void
+resurrect() : void
+clone() : Enemy*

Zombie

-weapon : Weapon*

+Zombie(other:Zombie&)
+operator=(other:Zombie&) : Zombie&
+~Zombie()
+update(tm:TileMap&, dt:float) : void
+attack() : void
+onPlayerContact(p:PlayerSoldier*) : void
+clone() : Enemy*

Owns Weapon* — implements Rule of Three
(copy ctor / operator= / dtor) so cloned
zombies don't share the same pistol.

Alien

+attack() : void

Martian

-phase : int
-hasPod : bool

+update(tm:TileMap&, dt:float) : void
+attack() : void
+onDeath() : void
+transitionToPhase2() : void
+clone() : Enemy*

Boss

#retreatThreshold : float
#hasRetreated : bool
#minionBatchSize : int

+Boss(scoreVal:int, hp:int, threshold:float, minionBatch:int)
+spawnMinions() : void
+virtual checkRetreat() : bool

IronNokana

-turretMissileCount : int
-hasFlamethrower : bool

+IronNokana()
+spawnMinions() : void
+checkRetreat() : bool
+attack() : void
+clone() : Enemy*

HairBusterRiberts

-rocketCount : int
-fireBombCount : int

+HairBusterRiberts()
+spawnMinions() : void
+checkRetreat() : bool
+attack() : void
+clone() : Enemy*

SeaSatan

-rocketCount : int

+SeaSatan()
+spawnMinions() : void
+checkRetreat() : bool
+attack() : void
+clone() : Enemy*

UltimateBoss

-currentForm : Boss*
-groundHP : int
-aerialHP : int
-aquaticHP : int
-formTimer : float
-formTransitionInterval : float
-groundForm : IronNokana*
-aerialForm : HairBusterRiberts*
-aquaticForm : SeaSatan*

+UltimateBoss()
+spawnMinions() : void
+checkRetreat() : bool
+transitionForm() : void
+attack() : void
+canUseForm(form:Boss*) : bool
+clone() : Enemy*

spawnMinions is pure-virtual: each concrete boss
spawns minions of its own type (Iron Nokana ->
ground infantry, Hairbuster -> aerial, Sea Satan
-> aquatic, UltimateBoss -> by current form).
checkRetreat has a default HP-fraction implementation
that UltimateBoss overrides for fused-form rules.

currentForm is a polymorphic Boss* — it points
at one of the three aggregated bosses below.
Form transitions just reseat the pointer.

Projectile

#owner : Entity*
#damage : int
#direction : coordSyst
#speed : float

+update(tm:TileMap&, dt:float) : void
+virtual onImpact() : void

StraightProjectile

#maxRange : float

+virtual update(tm:TileMap&, dt:float) : void

BallisticProjectile

#gravity : float
#initialSpeed : float

+virtual update(tm:TileMap&, dt:float) : void

Bullet

-isBentByFannum : bool

+update(tm:TileMap&, dt:float) : void
+bendTowardTarget(target:coordSyst, radius:float) : void
+onImpact() : void

Rocket

-blastRadius : float

+update(tm:TileMap&, dt:float) : void
+onImpact() : void

EnergyBeam

-range : float
-lifetime : float
-isInstantKill : bool

+update(tm:TileMap&, dt:float) : void
+render(rw:sf::RenderWindow&) : void
+onImpact() : void

GrenadeProjectile

-blastRadius : float

+update(tm:TileMap&, dt:float) : void
+onImpact() : void

FireBombProjectile

-poolRadius : float
-poolDuration : float

+update(tm:TileMap&, dt:float) : void
+onImpact() : void

BazookaShell

-blastRadius : float

+update(tm:TileMap&, dt:float) : void
+onImpact() : void

ReverseArcMissile

-blastRadius : float

+update(tm:TileMap&, dt:float) : void
+onImpact() : void

Explosion

-blastRadius : float
-damage : int
-lifetime : float

+update(tm:TileMap&, dt:float) : void
+render(rw:sf::RenderWindow&) : void

AreaHazard

#damagePerSecond : int
#duration : float

+virtual ~AreaHazard() = default
+update(tm:TileMap&, dt:float) : void
+render(rw:sf::RenderWindow&) : void
+isExpired() : bool

FirePool

-radius : float

+update(tm:TileMap&, dt:float) : void
+render(rw:sf::RenderWindow&) : void
+isExpired() : bool

FireStream

-streamLength : float
-direction : Direction*

+update(tm:TileMap&, dt:float) : void
+render(rw:sf::RenderWindow&) : void
+isExpired() : bool
+deactivate() : void

Weapon

#damage : int
#ammo : int
#fireRate : float
#maxAmmo : int

+virtual ~Weapon() = default
+fire(owner:Soldier*, aim:coordSyst) : Entity*
+consumeAmmo() : void
+clone() : Weapon*

ProjectileWeapon

#muzzleOffset : coordSyst

+virtual spawnProjectile(owner:Soldier*, aim:coordSyst) : Projectile*

Pistol

+fire(owner:Soldier*, aim:coordSyst) : Entity*
+clone() : Weapon*

Knife

-meleeRange : float
-meleeCooldown : float

+fire(owner:Soldier*, aim:coordSyst) : Entity*
+clone() : Weapon*

HeavyMachineGun

+fire(owner:Soldier*, aim:coordSyst) : Entity*
+clone() : Weapon*

RocketLauncher

-reloadTimer : float

+fire(owner:Soldier*, aim:coordSyst) : Entity*
+clone() : Weapon*

FlameShot

-streamLength : float
-activeStream : FireStream*

+clone() : Weapon*
+fire(owner:Soldier*, aim:coordSyst) : Entity*
+stopFire() : void

LaserGun

+fire(owner:Soldier*, aim:coordSyst) : Entity*
+clone() : Weapon*

Inventory

-equippedWeapon : Weapon*
-defaultPistol : Pistol*
-knife : Knife*
-grenadePrototype : Grenade*
-grenadeCount : int

+~Inventory()
+pickupWeapon(w:Weapon*) : void
+getCurrentWeapon() : Weapon*
+hasWeapon() : bool
+revertToPistol() : void
+setGrenadePrototype(g:Grenade*) : void
+getGrenadePrototype() : Grenade*
+getGrenadeCount() : int
+addGrenades(n:int) : void
+consumeGrenade() : Grenade*

grenadePrototype is the Prototype Pattern source.
consumeGrenade() returns a fresh clone for the
thrower to launch; the prototype itself is owned
by Inventory and persists.

Collectible

#consumed : bool

+virtual ~Collectible() = default
+onPickup(p:PlayerSoldier*, ps:PlayState&) : void
+isConsumed() : bool

Turkey

-saturationRestore : int

+onPickup(p:PlayerSoldier*, ps:PlayState&) : void

Fruit

-saturationRestore : int

+onPickup(p:PlayerSoldier*, ps:PlayState&) : void

POWPrisoner

+onPickup(p:PlayerSoldier*, ps:PlayState&) : void

SupplyCrate

-containedWeapon : Weapon*
-containedGrenades : int
-containedGrenadePrototype : Grenade*

+onPickup(p:PlayerSoldier*, ps:PlayState&) : void

InteractableObject

#interactionRadius : float
#isActive : bool

+virtual ~InteractableObject() = default
+onInteract(p:PlayerSoldier*, ps:PlayState&) : void
+canInteract(p:PlayerSoldier*) : bool

VehicleTerminal

-linkedVehicle : Vehicle*

+onInteract(p:PlayerSoldier*, ps:PlayState&) : void
+canInteract(p:PlayerSoldier*) : bool

CheckpointPost

-checkpointId : int

+onInteract(p:PlayerSoldier*, ps:PlayState&) : void
+canInteract(p:PlayerSoldier*) : bool

TileMap

-grid : Tile***
-width : int
-height : int
-seaLevel : int
-craters : Crater*
-craterCount : int

+getTile(x:int, y:int) : Tile*
+setTile(x:int, y:int, t:Tile*) : void
+destroyTilesInRadius(center:coordSyst, r:float) : void
+fillCraterWithWater(x:int, y:int) : void
+tickCraterFill(dt:float) : void
+getBiomeAt(x:int, y:int) : Biome*

TerrainChunk

-chunkX : int
-chunkWidthBlocks : int
-tiles : Tile**
-generated : bool

+generate(profile:NoiseProfile*) : void
+isLoaded() : bool

BiomeMapper

-aerialBiome : AerialBiome*
-plainsBiome : PlainsBiome*
-aquaticBiome : AquaticBiome*

+classify(height:float, profile:NoiseProfile*) : Biome*

RainSystem

-rainTimer : float
-isRaining : bool
-rainDuration : float
-triggerInterval : float

+update(tm:TileMap&, dt:float) : void
+triggerRain() : void
+isCurrentlyRaining() : bool

Camera

-viewCenter : coordSyst
-viewWidth : float
-viewHeight : float
-worldBounds : Box
-target : Entity*

+follow(target:Entity*) : void
+update(dt:float) : void
+applyToRenderWindow(rw:sf::RenderWindow&) : void
+isInView(hitbox:Box) : bool
+setWorldBounds(b:Box) : void
+clampToBounds() : void

InputHandler

-keyState : bool[256]
-mousePosition : coordSyst

+poll() : void
+isKeyPressed(key:int) : bool
+isKeyHeld(key:int) : bool
+getMousePosition() : coordSyst
+getAimAngle(playerPos:coordSyst) : float

Level

#tileMap : TileMap
#levelIndex : int

+virtual ~Level() = default
+virtual update(dt:float) : void
+virtual render(rw:sf::RenderWindow&) : void
+virtual generateTerrain() : void
+virtual spawnEnemies() : void
+virtual spawnPlayerVehicles() : void
+virtual isComplete() : bool
+getTileMap() : TileMap&

SurvivalLevel

-fixedSpawnsTable : EnemySpawn*
-nextSpawnIndex : int
-numOfSpawnBatches : int

+~SurvivalLevel()
+update(dt:float) : void
+render(rw:sf::RenderWindow&) : void
+generateTerrain() : void
+spawnEnemies() : void
+spawnPlayerVehicles() : void
+isComplete() : bool

BossLevel

-currentPhase : int
-groundBoss : IronNokana*
-aerialBoss : HairBusterRiberts*
-aquaticBoss : SeaSatan*
-ultimateBoss : UltimateBoss*

+update(dt:float) : void
+render(rw:sf::RenderWindow&) : void
+generateTerrain() : void
+spawnEnemies() : void
+spawnPlayerVehicles() : void
+isComplete() : bool
+advancePhase() : void
+triggerUltimateBossFusion() : void

CampaignLevel

-profile : NoiseProfile*
-chunks : TerrainChunk*
-chunkCount : int
-playerWorldX : int
-enemyKillCounts : int*
-numEnemyTypes : int
-vehicleKillCounts : int*
-numVehicleTypes : int

+update(dt:float) : void
+render(rw:sf::RenderWindow&) : void
+generateTerrain() : void
+spawnEnemies() : void
+spawnPlayerVehicles(): void
+isComplete() : bool
+streamChunkLeft() : void
+streamChunkRight() : void
+unloadFarChunks() : void

PerlinNoise

-seed : unsigned
-perm : int[512]

+initPerm() : void
+sample(x:float, y:float) : float
+fade(t:float) : float
+lerp(a:float, b:float, t:float) : float
+grad(hash:int, x:float, y:float) : float

FractalNoise

-perlin: PerlinNoise*

+sample(x:float, y:float, octaves:int, persistence:float, lacunarity:float) : float

NoiseProfile

#amplitude : float
#frequency : float
#octaves : int
#persistence : float
#lacunarity : float
#seed : unsigned
#aquaticCutoff : float
#plainsCutoff : float
#fractal : FractalNoise*

+virtual ~NoiseProfile() = default
+sample(x:float, y:float) : float
+getHeightAt(blockX:int) : float
+profileName() : char*
+getAquaticCutoff() : float
+getPlainsCutoff() : float

NormalProfile

+sample(x:float, y:float) : float
+getHeightAt(blockX:int) : float
+profileName() : char*

AmplifiedProfile

+sample(x:float, y:float) : float
+getHeightAt(blockX:int) : float
+profileName() : char*

FlatProfile

+sample(x:float, y:float) : float
+getHeightAt(blockX:int) : float
+profileName() : char*

NoiseProfileFactory

+createNormal(seed:unsigned) : NoiseProfile*
+createAmplified(seed:unsigned) : NoiseProfile*
+createFlat(seed:unsigned) : NoiseProfile*

ScoreManager

-currentScore : int

+addKillScore(e:Enemy*) : void
+addMeleeKillBonus() : void
+addAerialKillBonus() : void
+addMultiKillBonus(enemyCount:int) : void
+addMassacreBonus(enemyCount:int) : void
+addBossKillScore(isUltimate:bool) : void
+addLevelClearBonus(type:int) : void
+addFlawlessVictoryBonus() : void
+getScore() : int

HUD

-font : sf::Font

+render(rw:sf::RenderWindow&, p:PlayerSoldier*, sm:ScoreManager&) : void
+renderDamageHue(rw:sf::RenderWindow&, p:PlayerSoldier*) : void
+renderBestPlayerOverlay(rw:sf::RenderWindow&, ghost:PlayerSoldier*) : void
+renderGenerationOverlay(rw:sf::RenderWindow&, gen:int) : void

MainMenu

-selectedMode : GameMode*
-selectedProfile : NoiseProfile*
-font : sf::Font

+render(rw:sf::RenderWindow&) : void
+handleInput(in:InputHandler&) : void
+getSelectedMode() : GameMode*
+getSelectedProfile() : NoiseProfile*
+isReady() : bool

CollisionManager

+update(roster:Entity**, n:int, tm:TileMap&) : void
+resolveAABB(a:Box, b:Box) : bool
+handleProjectileVsLiving(p:Projectile*, le:DamagableEntity*) : void
+handleMeleeVsEnemy(p:PlayerSoldier*, e:Enemy*) : void
+handleVehicleVsEntity(v:Vehicle*, e:Entity*) : void
+handleExplosionVsEntities(ex:Explosion*, roster:Entity**, n:int) : void
+handleCollectibleVsPlayer(c:Collectible*, p:PlayerSoldier*) : void
+handleEntityVsWorld(e:Entity*, tm:TileMap&) : void
+handleEntityVsWater(e:Entity*, tm:TileMap&) : void
+applyFannumTax(bullets:Projectile**, n:int, fatPlayer:PlayerSoldier*) : void

SoundManager

-currentMusic : sf::Music*
-soundBuffers : sf::SoundBuffer*[32]
-activeSounds : sf::Sound*[16]
-soundCount : int
-isUnderwater : bool

+playSound(id:int) : void
+playMusic(path:char*) : void
+setUnderwater(underwater:bool) : void
+applyMuffleFilter() : void
+decomposeFrequencies(buffer:sf::SoundBuffer*) : void
+update() : void

VoiceActivationEngine

-referenceMatrix : float**
-matrixRows : int
-matrixCols : int
-similarityThreshold : float
-isActivated : bool

+start() : void
+pollMicrophone() : void
+loadReferenceAudio(path:char*) : void
+decomposeFFT(samples:float*, n:int) : float**
+computeCosineSimilarity(a:float, b:float, rows:int, cols:int) : float
+listen() : bool
+isCommandDetected() : bool

NeuralNetwork

-nodes : Node**
-nodeCount : int
-evaluationOrder : int*

+buildFromGenome(g:Genome*) : void
+feedForward(inputs:float*, n:int) : float*
+getOutput(index:int) : float

Genome

-nodes : Node**
-nodeCount : int
-connectionGenes : ConnectionGene*
-connectionCount : int
-network : NeuralNetwork*
-fitness : float
-speciesId : int

+evaluate(ps:PlayState&) : float
+crossover(other:Genome*) : Genome*
+mutateAddNode() : void
+mutateAddConnection() : void
+mutateWeights() : void
+mutateToggleConnection() : void
+compatibilityDistance(other:Genome*) : float
+clone() : Genome*
+serialize(out:FILE*) : void
+deserialize(in:FILE*) : void

Species

-representative : Genome*
-members : Genome**
-memberCount : int
-bestFitness : float
-staleness : int

+addMember(g:Genome*) : void
+clear() : void
+chooseRepresentative() : void
+getAdjustedFitnessSum() : float

NEATManager

-population : Genome**
-populationSize : int
-speciesList : Species**
-speciesCount : int
-generationNumber : int
-globalBest : Genome*
-globalBestFitness : float
-compatibilityThreshold : float
-c1 : float
-c2 : float
-c3 : float
-innovationCounter : int

+initPopulation(size:int) : void
+evaluateGeneration(ps:PlayState&) : void
+speciate() : void
+adjustFitness() : void
+cullSpecies() : void
+reproduce() : void
+evolve() : void
+getGlobalBest() : Genome*
+saveGlobalBest(path:char*) : void
+loadGlobalBest(path:char*) : void
+getCurrentGeneration() : int
+nextInnovationNumber() : int

EnemySpawner

-masterCopies : Enemy**
-copyCount : int
-spawnWeights : float*
-heldHeavySpawnTimer : float

+spawn(level:Level&) : Enemy*
+adjustWeights(deltas:float*) : void
+queryAboutToSpawnHeavy() : bool
+holdNextHeavySpawn(seconds:float) : void

TimelineManager

-snapshots : Snapshot*
-snapshotCount : int
-timelineBranches : TimelineNode*

+takeSnapshot() : void
+rewindLast10Seconds() : void
+branchTimeline() : void
+switchToBranch(nodeId:int) : void
+saveToFile(path:char*) : void
+loadFromFile(path:char*) : void

FingerprintRecorder

-parameters : FingerprintParam*
-parameterCount : int

+getParam(id:char*) : FingerprintParam*
+setParamValue(id:char*, value:float) : void
+recordAction(action:PlayerAction*, ctx:PlayerSoldier*) : void
+saveToCSV(path:char*) : bool
+loadFromCSV(path:char*) : bool

FingerprintAutopilot

-profile : FingerprintRecorder*
-remainingTime : float

+tick(dt:float, p:PlayerSoldier*) : void
+isExpired() : bool

DeathCompositor

-deathFingerprints : DeathFingerprint*
-fingerprintCount : int

+recordDeath(p:PlayerSoldier*, enemies:Enemy**, n:int) : void
+matchCurrentState(p:PlayerSoldier*) : float
+tiltSpawnWeights(spawner:EnemySpawner*) : void
+saveToFile(path:char*) : void
+loadFromFile(path:char*) : void

RagebaitEngine

-runHistory : RunRecord*
-runHistoryCount : int
-expectedScore : int
-midpointScore : int
-targetGap : float

+recordRunMidpoint(score:int) : void
+recordRunFinal(score:int) : void
+updateExpectation() : void
+shouldDowngradeSpawn() : bool
+saveToFile(path:char*) : void
+loadFromFile(path:char*) : void

ChainOfDeath

-currentLabel : BehaviourLabel*
-labels : BehaviourLabel**
-labelCount : int
-transitionTable : float**
-historicalTable : float**
-recencyWeight : float
-inputWindow : PlayerInputRecord*
-inputWindowSize : int

+inferCurrentState(inputs:PlayerInputRecord*, n:int) : BehaviourLabel*
+predictNextState() : BehaviourLabel*
+updateTransition(from:BehaviourLabel*, to:BehaviourLabel*) : void
+mergeWithHistory() : void
+saveToFile(path:char*) : void
+loadFromFile(path:char*) : void

UrduAssemblyTerminal

-isActive : bool
-inputBuffer : char[256]
-bufferPos : int

+activate() : void
+deactivate() : void
+parseAndExecute(ps:PlayState&) : void
+parseInstruction(line:char*) : Instruction*
+render(rw:sf::RenderWindow&) : void
+handleKeyInput(key:int) : void

owner

1

after landing

0..1

morphs into on landing

currentForm pointer

0..1

owns

1

owns stack

*

owns

1

owns

1

owns active mode (Strategy)

1

owns & deletes

*

creates & destroys

0..1

owns active state

1

owns facing

1

1 owns AI logic

owns weapon 0..1

owns 1

equipped

0..1

default

1

melee

1

grenade prototype

1

clones prototype on throw

owns staging state

1

owns active mode

1

label set

*

input window

*

recorded actions

*

* creates per line parsed program owns master copies

*

owns table

*

contained

0..1

contained prototype

0..1

owns 1

owns chunks

*

owns

1

owns grid

*

tracks

*

owns chunk tiles

*

owns

1

owns

1

owns flyweight

1

owns flyweight

1

owns flyweight

1

active stream

0..1

creates

owns direction

1

phenotype

1

node genes

*

connection genes

*

computation graph

*

population

*

globalBest (cloned)

0..1 species list

*

parameters

*

history

*

snapshots

*

branch root

1

fingerprints

*

1

1

1

1

1

1

1

1

1

0..1

1

1

1

1

1

1

startup trigger

1

owns evolver 1

1

reads selection

cause ref 0..1

death-fingerprint ref (TA spec)

0..1

pilot

0..1

references

*

fusion members

2..4

linkedPlayer

1

uses profile

1

drives from

1

follows

0..1

follows

ground form

1

delegates ground attack

aerial form

1

delegates aerial attack

aquatic form

1

delegates aquatic attack

retreat tracker

0..1

retreat tracker

0..1

retreat tracker

0..1

phase 4

0..1

linked

1

master copy ref

1

biome ref 1

1 selected

selected (campaign only)

1

creates (fire)

creates

takeDamage

constructs per hit

reads radius

applies dps

calls blockAttack

checks fire/explosion rule

vehicle vs entity

mutates

reads

reads score

reads hueAlpha()

reads scoreValue

creates

reads cutoffs

classifies tiles

flips magnet
spawns on impact spawns on impact

spawns on impact

destroys via Tile::onExplosion

only fire/explosion kills only fire/explosion kills

drives input

produces commands

tilts weights downgrades spawns

informs spawner

snapshots state

fires in phase 1

fills craters

creates new water tiles

spawns on release

Fannum Tax bending

manages boss phases

assigns to PlayState

uses pistol

checks submersion

runs simulation

evaluates fitness

modifies stats

modifies score

spawns into level

replays commands